

Blind Spots in Blind Signatures: A System-Level Security Analysis of Deployed Chaumian Ecash

Huaifeng Chen¹, Yuchang Zhang¹, and Yu Cheng²

¹ North China Institute of Computer Systems Engineering
chenhf@ncse.com.cn, zhangyuchang26@mails.ucas.ac.cn

² Beijing Electronic Science and Technology Institute
chengyu_cs@zjut.edu.cn

Abstract. Cashu, the dominant deployed Chaumian ecash system, has grown to 31 specification documents, 58 indexed mints, and seven interoperable wallets, while the cryptographic literature has studied only its blind-signature core. Every incident in the system’s short audited history, including a cross-mint theft disclosed in early 2026, is compositional: it emerges from keyset rotation, wallet recovery, and multi-mint identity. We give the first system-level security analysis of a deployed ecash system, built on a five-layer model, a nine-property family (three properties new), and a stateful one-more-unforgeability theorem whose proof separates forgery from theft; the deployed attacks commit theft without forging anything. We reproduce the disclosed theft end-to-end, exhibit a strictly stronger public-key-claim variant, and measure all 58 indexed mints: every reachable mint declares support for DLEQ proofs, the sole remaining defense, yet declarations are unauthenticated—the guardrail sits with the issuer, not the wallet. A theft theorem gives necessary and sufficient conditions for prevention, and replaying three fixes shows the cheapest one costs honest issuers nothing. A defect taxonomy and a three-system comparison suggest a broader lesson: recoverability is not free. All artifacts are pure standard-library Python and pass the 49 official test vectors.

Keywords: Chaumian ecash · blind signatures · Cashu · system-level security analysis · mainnet measurement.

1 Introduction

Cryptographic primitives earn trust through theorems; deployed systems earn it, and lose it, through composition. Nowhere is the gap more visible today than in the revival of Chaumian ecash. Regulatory pressure on on-chain privacy tools has redirected demand toward custodial-but-blind alternatives: mints that issue blind-signature tokens against Bitcoin deposits. The dominant open instantiation, Cashu, has grown from a single cryptographic note into a family of 31 specification documents (NUT-00–30) covering keyset rotation, deterministic wallet

recovery, HTLC locks, offline validation, blind authentication, and multi-mint tokens; at the time of writing, 58 mints are indexed with 604 community-signed reviews and seven wallets interoperate across them [1,2]. The academic response, however, has studied one layer: the blind-signature primitive at the core, which sits in an active literature of one-more-unforgeability gradations [6], threshold blind BLS and OPRFs [7], round-optimal threshold blind signatures [8], and new pairing-free constructions [9].

Meanwhile the deployed system’s short audited history reads as a catalogue of what those theorems cannot see. A cross-mint theft, disclosed in January 2026 with a bug bounty and no academic follow-up to date, exploited the interaction of a truncated keyset identifier in the deterministic-recovery standard, BIP32 path aliasing, and an unauthenticated signature-restore endpoint [3]. A denial-of-service CVE attacked mints that accepted unbounded HTLC preimages the specification had bounded only in prose [4]. A hash-to-curve domain-separation flaw was fixed silently in 2024 through implementation commits, with no advisory [5]. Not one of these is expressible as a failure of the blind-signature primitive. They are blind spots of blind signatures—failures of the *system* the signature lives in.

This paper delivers the first system-level security analysis of a deployed ecash system, in the break–fix–verify tradition established for deployed payment standards [10,11]. Our contributions:

1. **A five-layer model and a property family with three new properties.** We formalize Cashu as the composition of token, keyset, spending-condition, offline-validation, and multi-mint layers (section 2), and define P1–P9, including *restore read-onlyness* (P6), *derivation binding* (P8), and *cross-mint theft prevention* (P9), none of which appear in prior literature. An executable model validates P1/P2/P4/P6/P7 over randomized traces with zero violations.
2. **A stateful unforgeability theorem.** Our StateOMU game adds three stateful oracles (keyset-rotation timing, restore replay, wallet corruption) to classical one-more unforgeability (sections 4 and 5). Its proof separates forgery-resistance from theft-resistance: the deployed attack family, and our stronger variant below, commit only the latter.
3. **A stronger attack, and a measurement that relocates the threat.** We reproduce the disclosed theft end-to-end and exhibit a *public-key-claim* variant that converts the victim’s unblinding into proofs directly spendable at the honest mint, passes keyset-identifier verification, and requires no signature conversion; its precondition chain (DLEQ optionality, wallet behavior on absent proofs, missing key-ownership checks) we verify against specification text and two reference implementations (section 3). Our mainnet measurement of all 58 indexed mints then shows that every reachable mint *declares* DLEQ support, while declarations are unauthenticated and a malicious mint may declare without delivering. The remaining defense thus rests on issuer goodwill rather than client enforcement.

4. **A necessary-and-sufficient theft theorem and replay-verified fixes.** Theorem 3 characterizes cross-mint theft prevention exactly: safe if and only if derivation binding, wallet-mandatory DLEQ, or registration-time key ownership holds. We implement all three fixes in the executable model, replay the attack (baseline 4/4 theft; all fixes 0), and confirm the honest flow survives (section 6).
5. **A defect taxonomy and a cross-system perspective.** Ten audited findings partition into specification-, implementation-, and composition-level classes with action-guiding detectability (section 7); a three-system comparison shows the property family separating Cashu (violated then repaired), Fedimint (design-level defense), and Privacy Pass (vacuous, because it forgoes recovery) (section 8): every recovery capability arrives with new security obligations.

All artifacts are pure standard-library Python, pass all 49 official test-vector checks, and reproduce every reported experiment.

2 System Model and Properties

2.1 Cashu in One View

A mint publishes per-amount keys $K_s = k_s G$ grouped into *keysets*, each with an identifier *id* (legacy V1: 8 bytes; current V2: 33 bytes derived from the keys, unit, fees, and expiry), an active flag, and an optional final expiry. A user blinds $Y = \text{h2c}(x)$ as $B = Y + rG$, receives the blind signature $C' = k_s B$, and unblinds $C = C' - rK_s$; the pair (x, C) is a bearer proof. The mint verifies $C \equiv k_s \text{h2c}(x)$ and records x in its spent-secrets ledger. Around this core, NUT-03/04/05 implement swap, mint, and melt; NUT-09 restores historical blind signatures from blinded messages; NUT-10/11/14 attach spending conditions; NUT-12 provides DLEQ proofs for offline validation; NUT-13 derives (x, r) deterministically from a BIP32 seed, with version-dependent paths; NUT-22 issues blind authentication tokens as auth-unit ecash.

2.2 Five-Layer Composition

We model the system as the synchronous composition of five lifecycle layers: *token* (unborn/blinded/signed/owned/spent), *keyset* (active/inactive/expired, per-keyset fees), *spending conditions* (witnesses, P2PK, HTLC preimages), *offline validation* (DLEQ carried in signatures and proofs), and *multi-mint interaction* (keyset-identifier namespaces, cross-mint tokens, restore endpoints). The executable model (MintSim) implements the six operations with the preconditions and effects of Table 1; Figure 1 juxtaposes the five layers with the blind-signature core they wrap.

Over 30 randomized traces (312 operations) the model satisfies conservation, no-double-spend, keyset binding, restore read-onlyness, and expiry safety with zero violations; replayed and cross-keyset replays of a spent secret are rejected

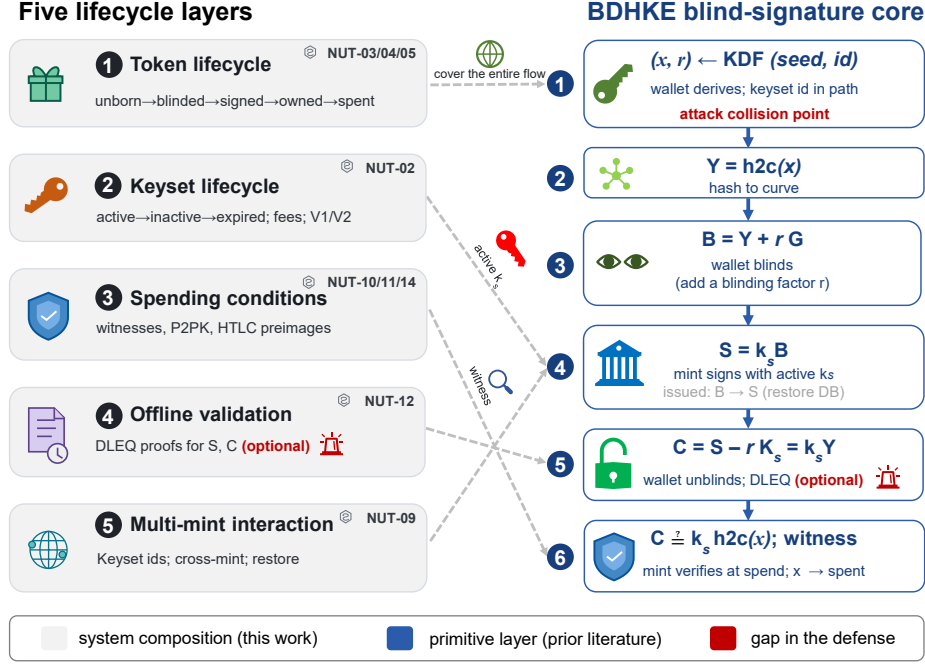


Fig. 1. Five-layer lifecycle composition of the Cashu system around its BDHKE blind-signature core (section 2). Left: the five lifecycle layers (1)–(5) with their governing NUT documents. Right: the core data flow from derived secrets to spend-time verification (notation as in section 2); the restore database ($B \mapsto S$) and the optional DLEQ check attach at issuance and acceptance. The primitive literature studies only the right column; every incident analyzed in this paper (sections 3 and 7) originates in the left.

(P2), and outputs to inactive keysets are refused (P4). Fee-aware conservation matches the reference implementation’s balance check ($\sum out \leq \sum in - \sum [ppk \cdot s/1000]$).

2.3 Property Family

Table 2 summarizes P1–P9; three properties are new.

The spent-secrets domain (O1). The modern hash-to-curve is identifier-independent; the same secret x maps to the same point under *every* keyset of every mint. A per-keyset spent ledger would therefore permit replaying one unspent proof across keysets. The swap specification never states the domain; the reference implementation’s input-verification routine is documented *not* to check spentness, delegating it to the database state machine. Our model adopts, and our P2 experiments confirm, a *global* domain, and theorem 1 uses it essentially (Case ii-a of the proof). We propose MUST-level language making the global domain normative.

Table 1. Executable model operations.

Op	Precondition	Effect
mint	quote paid; outputs in active keysets	issued += outputs
swap	inputs verify, unspent, unexpired; $\sum out \leq \sum in - fees$; outputs active	spent += input secrets; sign
melt	as swap inputs	external payment
restore	—	return existing issued entries (read-only)
rotate	new keyset active	old keyset $\rightarrow$ inactive
expire	$now > final_expiry$	reject inputs; clear nullifiers

Table 2. Property family P1–P9. * = new in this work.

ID	Property	Formal anchor
P1	Conservation	$\sum in = \sum out + \sum fees$
P2	No double-spend	global spent-secrets dedup
P3	No inflation	StateOMU (section 5)
P4	Keyset binding	inactive-in/active-out
P5	Unlinkability	composite blindness (Prop. 1)
P6*	Restore read-onliness	oracle purity
P7	Expiry safety	nullifier clearing
P8*	Derivation binding	PRF domain separation
P9*	Cross-mint theft prevention	Thm. 3 conditions

3 Attacks and Measurement

3.1 Reproducing the Residue-Collision Theft

We reproduce the disclosed NUT-13 attack end-to-end in scaled parameters (path modulus 2^{12} for demonstration; the derivation algebra is identical to the real attack at $2^{31} - 1$, whose cost is attested by the original disclosure’s brute-force experiments). A malicious mint constructs a keyset whose identifier residue collides with a target mint’s; a NUT-13 wallet then reuses identical (x, r) pairs across both mints, so the victim’s swap outputs equal blinded messages already signed by the target mint, which the unauthenticated restore endpoint hands to the attacker. In our reproduction: 8/8 blinded-message reuse, 8/8 restore hits [3]. The victim-visible timeline is quiet until it is complete: an airdropped token, a seemingly successful withdrawal (faked), and only later an honest-mint payment failing with “proof already spent.”

3.2 A Stronger Variant: the Public-Key-Claim Attack

Attack. The malicious mint’s keyset includes the *target mint’s* public key K as its own per-amount key (alongside one self-generated key to differentiate the

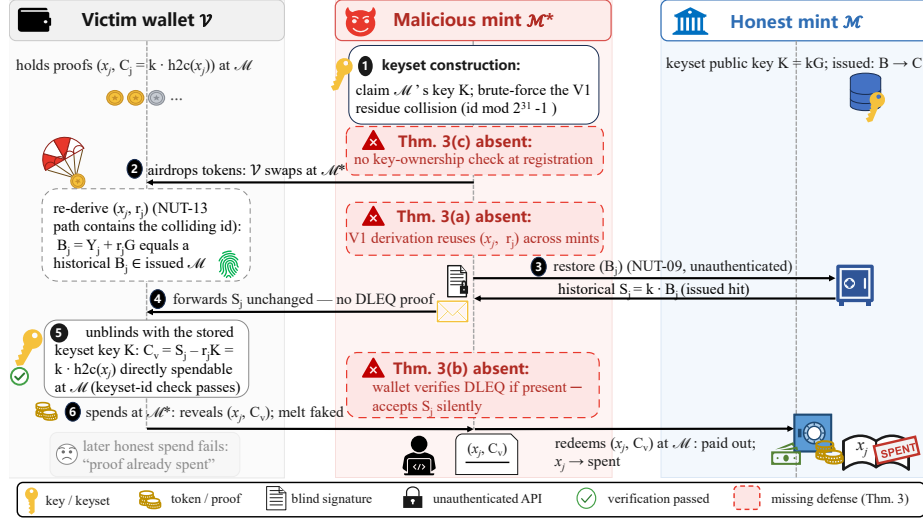


Fig. 2. The public-key-claim attack (section 3; end-to-end reproduction EXP-004, 8/8 stolen proofs). Three parties: victim wallet $\mathcal{V}$, malicious mint $\mathcal{M}^*$, and honest mint $\mathcal{M}$ with keyset key $K = kG$; S_j denotes the blind signature on B_j . (1) Colliding keyset construction claiming K ; (2) airdrop and swap re-derive (x_j, r_j) , colliding with history; (3) the unauthenticated NUT-09 restore returns $S_j = k B_j$; (4) S_j is forwarded without a DLEQ proof; (5) the victim unblinds with the claimed key to $C_v = k h2c(x_j)$, directly spendable at $\mathcal{M}$; (6) spending reveals (x_j, C_v) , which $\mathcal{M}^*$ redeems. Dashed red boxes mark the three defenses of theorem 3—derivation binding (a), wallet-mandatory DLEQ (b), registration key ownership (c)—all absent in the deployed configuration.

identifier), and forwards the restored signature S unmodified. The victim unblinds with K , the key its wallet stored for that keyset, computing

$$C_v = S - rK = k h2c(x) + krG - krG = k h2c(x),$$

a proof *directly spendable at the honest mint* (Figure 2 traces the six-step chain). When the victim spends at the malicious mint (whose melt endpoints simulate success), the attacker banks a valid target-mint proof and redeems it; the victim’s next honest spend fails. In our reproduction all eight stolen proofs equal $k h2c(x_j)$ exactly; in the original variant’s algebra (returned under a self-owned key $A = aG$) we find no closed-form conversion of the unblinded value to $k h2c(x)$ —the original attacker must fall back to candidate testing at the honest mint. Table 3 compares the phases.

Preconditions, verified against two implementations. (P1) NUT-12 is optional and binds wallets only “if included”: the specification permits mints to omit DLEQ entirely. (P2) The reference Python wallet verifies DLEQ proofs “if present”; signatures without proofs are accepted silently. (P3) Unblinding uses the wallet-stored keyset key; neither implementation we examined re-derives the keyset identifier from published keys or demands proof of key ownership.

Table 3. Attack-phase comparison. “Recovery” = obtaining an honest-mint-valid proof.

Phase	Disclosed variant	Public-key-claim	Anchor
Collision	$\Theta(2^{31})$ trials	$\Theta(2^{31})$ trials	both
Reuse	8/8 identical B	8/8 identical B	EXP-004
Restore oracle	8/8 hits	8/8 hits	EXP-004
Mint key material	own key a	none (claims K)	—
Keyset-ID check	bypassed (unverified)	<i>passes</i>	§3
Recovery	candidate testing	direct ($C_v = kh2c(x)$)	EXP-004

The variant thus requires all three conditions, and each holds in the examined ecosystem today.

3.3 Mainnet Measurement: the Guardrail Is on the Wrong Side

If the attack’s only defense is DLEQ, how much of the deployed ecosystem actually wields it? We measured all 58 mints indexed by the community directory (fetched 2026-09-10), querying each mint’s public, unauthenticated `/v1/info` endpoint (NUT-06) serially with an academic user agent and rate limiting: a non-invasive capability census. Of 58, 29 were reachable (Figure 3; the remainder timed out or rejected connections; we report proportions over reachable mints and publish the full per-mint matrix; a second sweep on a different day reproduced every figure). The unreachable half decomposes into 14 connection or DNS failures, 11 HTTP errors, 3 non-JSON responses, and 1 timeout, consistent with churned or endpoint-migrated mints rather than selective blocking. Among reachable mints, 29/29 (100%) declare NUT-12 support and 28/29 (96.6%) expose the restore endpoint (NUT-09) the attack chain requires; no reachable mint pairs that endpoint with absent DLEQ, so the exposure lies in what an unauthenticated declaration cannot rule out.

These declarations are self-reported and unauthenticated: nothing prevents a malicious mint from declaring NUT-12 support and returning signatures without proofs, which is precondition (P2) above, verified in wallet code. The defense therefore rests on the issuer’s goodwill alone—which is why section 6 moves the check into the wallet, where it costs honest issuers nothing.

3.4 Specification–Vector Inconsistencies

Two previously unreported defects in the official test vectors: the NUT-12 proof-level DLEQ vector fails verification under all six reasonable interpretations of the Carol-side reconstruction while every other vector in the same file passes; and the TokenV3 JSON example contradicts its own base64 serialization in one hexadecimal digit of a signature value (both are valid curve points). We have drafted responsible-disclosure reports.

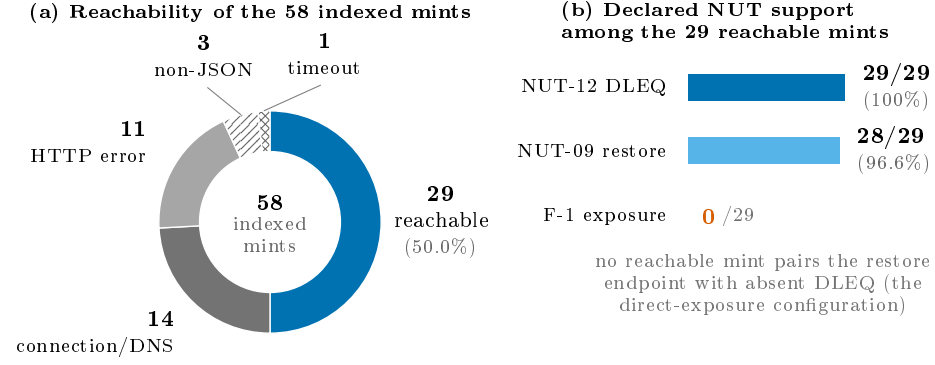


Fig. 3. Mainnet capability census of all 58 indexed mints (EXP-011; section 3.3). (a) Reachability: 29/58 reachable (50.0%); unreachable: connection/DNS (14), HTTP errors (11), non-JSON (3), timeout (1). (b) Declared support among the 29 reachable mints: NUT-12 DLEQ 29/29 (100%), NUT-09 restore 28/29 (96.6%), direct F-1 exposure 0/29—declarations are self-reported and unauthenticated.

4 Security Definitions

Definition 1 (StateOMU). *The game pits a PPT adversary $\mathcal{A}$ against a mint holding a keyset sequence $K_1, \dots, K_\rho$ with per-amount keys, a global ledger $\text{spent} \subseteq \{0, 1\}^*$, a restore database issued : $B \mapsto C'$, and initial balance β_0 . $\mathcal{A}$ accesses four oracles:*

- $\mathcal{O}_{\text{Sign}}(K, s, B)$: returns $k_{K,s}B$ for active K , recording issued;
- $\mathcal{O}_{\text{Spend}}(K, s, x, C)$: accepts iff $C = k_{K,s}\text{h2c}(x)$, $x \notin \text{spent}$, and K unexpired; then $x \mapsto \text{spent}$;
- $\mathcal{O}_{\text{Restore}}(\cdot)$: returns existing issued entries to any caller;
- wallet corruption: reveals deterministic (x, r) state.

$\mathcal{A}$ wins if it outputs proofs totaling more than β_0 plus all $\mathcal{O}_{\text{Sign}}$ -issued value. Cashu is StateOMU-secure if every PPT $\mathcal{A}$ wins with negligible probability.

Adversary tiers: **E1** malicious user (targets P3); **E2** malicious mint (targets P5/P9); **E3** network observer (targets P5); **E4** = E2+E3 with deterministic-derivation knowledge (targets P8/P9).

5 Theorems

Lemma 1 (Blinding intractability). *Given (G, K, B, kB) with $B = Y + rG$ for unknown r , computing kY is as hard as solving DDH on $(G, K, B - Y, kB - kY)$.*

Theorem 1 (StateOMU). *In the random-oracle model, under DDH, if the state machine satisfies P1, P2, and P4, then Cashu satisfies StateOMU: for*

every PPT adversary $\mathcal{A}$ making q_h hash queries, $\Pr[\mathcal{A} \text{ wins}] \leq \text{Adv}^{\text{DDH}} + q_h \cdot 2^{-\lambda} + \text{negl}(\lambda)$.

Lemma 2 (Capability monotonicity). *Rotation and expiry only remove oracle interfaces. For any adversary, the winning probability under the restricted interface is bounded by that under the unrestricted one.*

Proof. Both operations induce game transformations whose trace sets are subsets (refused calls terminate transcripts); probabilities over a subset of transcripts are bounded by probabilities over the superset.

Lemma 3 (Restore absorption). *If P6 holds, every $\mathcal{O}_{\text{Restore}}$ response is a deterministic function of the $\mathcal{O}_{\text{Sign}}$ transcript, and $\mathcal{O}_{\text{Restore}}$ can be simulated and dropped without changing the winning probability.*

Proof. P6 (validated over all randomized model traces) states that restore returns only entries of **issued** and never mutates **spent**; the simulator forwards the corresponding transcript entries.

Lemma 4 (Accounting attribution). *If $\mathcal{A}$ wins, some proof (x^*, C^*) in its output is accepted by $\mathcal{O}_{\text{Spend}}$ yet uncovered by any $\mathcal{O}_{\text{Sign}}$ call of $\mathcal{A}$.*

Proof. The output is a finite multiset; subtracting, via the P1 identity, the value covered by $\mathcal{O}_{\text{Sign}}$ -accounted proofs from the total leaves a strictly positive residual carried by at least one element.

Proof (Proof of theorem 1). By lemmas 2 and 3 we analyze the game with restore simulated from the signing transcript. Let (x^*, C^*) be the witness of lemma 4. The reduction embeds a DDH challenge $(G, K^\dagger, Y^\dagger, T)$ by programming one hash-to-curve response (the anchor $\hat{x}$, uniform among q_h queries) to $Y^\dagger$ and one keyset-amount key to $K^\dagger$. Let (x^*, C^*) be the surplus witness. Since $\mathcal{O}_{\text{Spend}}$ accepts, $C^* = kY(x^*)$.

Case (i): for all r , the message $B = Y(x^*) + rG$ is absent from **issued**. Then obtaining C^* uses no oracle path; if $x^* = \hat{x}$ the reduction outputs C^* as its DDH answer (correct exactly when the challenge is a DH tuple). The hybrid interpolates over the q_h candidate anchors: for each j , the first j hash responses are programmed as uniform points and the $(j+1)$ -th carries the challenge; neighboring games differ only when the adversary queries exactly the $(j+1)$ -th fresh input, an event of probability at most $2^{-\lambda}$ per query, giving total loss $q_h \cdot 2^{-\lambda}$.

Case (ii): some $B^* = Y(x^*) + r^*G \in \text{issued}$. By lemma 1, producing C^* without r^* is equivalent to DDH, so $\mathcal{A}$ holds r^* . Sub-case (ii-a): r^* was learned by wallet corruption (tier E4). The adversary may indeed cause the same (x^*, r^*) to be signed under several colliding keysets (the legacy V1 residue class), unblinding each to $C_j = k_j Y(x^*)$ —but every such proof shares the secret x^* , and by P2 (global deduplication) only the first redemption is accepted; the duplicates carry zero residual value, contradicting surplus. Sub-case (ii-b): r^* was self-chosen and submitted via $\mathcal{O}_{\text{Sign}}$; the proof is covered by that call’s accounting. Both sub-cases contradict the witness; only the DDH solver remains.

Theorem 2 (Derivation binding). *(a) Under the legacy V1 mapping, keyset-identifier residues collide after $\Theta(2^{31})$ targeted trials, and derivation binding fails. (b) Under the V2 HMAC derivation, distinguishing the outputs of two keysets from independent PRF outputs implies a PRF distinguisher; derivation binding holds.*

Proof (Proof sketch). (a) The V1 path compresses the identifier to $id \bmod (2^{31}-1)$; a malicious mint draws keys until its identifier lands in the target residue class, at expected cost 2^{31} hash evaluations. The derivation paths, hence (x, r) , then coincide. (b) The V2 KDF computes (x, r) as HMAC outputs over (seed, full identifier, counter); any adversary deriving information about one keyset’s outputs from another’s yields a PRF distinguisher. Our experiments confirm (a) end-to-end at scaled parameters and (b) over 3,000 keyset pairs with zero collisions.

Theorem 3 (Cross-mint theft prevention, iff). *Under DDH, an E_4 adversary can be prevented from converting a victim’s derived secrets into a valid unspent proof of the honest mint if and only if at least one of the following holds: (a) P8 derivation binding; (b) wallets reject DLEQ-less blind signatures from mints that advertise NUT-12; (c) keyset registration includes key-ownership verification.*

Proof (Proof sketch). Sufficiency: under (a), the secret was never derived for the malicious keyset, so no signed blinded message contains its point, reducing to Case (i) of theorem 1; under (b), the forwarded signature lacks a DLEQ proof under the claimed key, and producing one requires its discrete log (soundness of Chaum–Pedersen proofs); under (c), the claimed identifier must match the hash of published keys and the registrant must prove ownership of each key, which the attacker cannot do for K . Necessity: the public-key-claim attack of section 3 is a complete construction when none of (a)–(c) holds—we execute it end-to-end (baseline 4/4 theft). Necessity is constructive—the attack above—rather than an optimality bound.

Proposition 1 (Composite unlinkability). *For any polynomial operation sequence, no PPT distinguisher links the mint’s view $\{(B_i, C'_i)\}$ to a spent proof (x, C) with advantage beyond DDH, provided blinding factors remain secret.*

Proof (Proof sketch). Induction over the operation sequence: the base case is single-issuance blindness ($C = C' - rK$ re-randomizes a DH tuple); each step adds a fresh blinded message, a restore response (a function of prior views, restore absorption), or the challenge itself—no step introduces a new algebraic relation between views and challenges. Numerically, the composite distinguisher’s advantage over 300 trials stays within the two-sigma noise band of a fair coin (0.040 single-step, 0.023 composite), rising to certainty once a blinding factor is revealed—the privacy face of the same deterministic-derivation weakness that enables the thefts.

Table 4. Fix verification: attack replay and honest-flow survival.

Configuration	Attack outcome	Honest flow
No fix (baseline)	4/4 theft	—
Fix-A: wallet-mandatory DLEQ	0/4 (forged proofs negligible)	survives
Fix-B: key ownership at registration	registration refused; forgery 0/10 ⁴	survives
Fix-C: V2 derivation	0/4 (no reuse; restore empty)	survives

Orthogonality. Theorem 1 and theorem 3 concern disjoint adversaries and disjoint losses. The NUT-13 family and our variant mint *no value*; they redirect proofs of the victim’s existing balance, so the StateOMU accounting never registers a surplus—the game cannot even express the loss, since Case (ii-a) explicitly absorbs multi-keyset re-derivations under P2. Conversely, theft prevention says nothing about forging fresh value. Prior treatments conflate the two under “security of ecash”; separating them shows why the primitive-level guarantees did not stop the deployed incidents, and what a fix must add.

6 Verified Fixes

We implement three repairs in the executable model and replay the attack; Table 4 reports results.

Fix-A and the zero-cost policy argument. Fix-A is a one-line normative change: wallets MUST reject blind signatures lacking a DLEQ proof from any mint whose **info** advertises NUT-12. Our measurement (section 3.3) shows every reachable mint already declares NUT-12 support, so no honest issuer changes behavior and the migration cost for issuers is zero. Wallets require a software update, and mints that do not advertise NUT-12 are unaffected, since the MUST binds only advertised capability; only the attacker’s declare-but-don’t-deliver path is closed. The lesson generalizes a pattern from the EMV line [10]: when compliance is already universal, upgrading a SHOULD to a wallet-side MUST is the cheapest security a specification can buy. Fix-B (ownership proofs at registration) is the stronger structural repair but requires protocol change; a random-forgery sweep against its Chaum–Pedersen check accepts 0 of 10⁴ attempts, matching the 2^{-256} soundness bound; Fix-C is already shipped as NUT-13 v2; in the replay it breaks the attack at its first link (no blinded-message reuse, empty restore response).

7 Defect Taxonomy

Table 5 collects the ten audited findings with their classes and evidence. Three readings follow. First, every incident is compositional or specification-level; none

Table 5. Audited findings. Spec=specification-level, Impl=implementation-level, Comp=composition-level. † = new in this work.

#	Class	Finding (evidence)
O1	Spec	Spent-secrets dedup domain unspecified; reference <code>verify_inputs</code> documented not to check it
O2	Spec	DLEQ nonce-counter exhaustion (256) behavior undefined
O3	Spec	Restore endpoint: no authentication or rate-limit norm
O4†	Spec	NUT-12 proof-level vector inconsistent with specification text (six interpretations fail; all other vectors pass)
O5†	Spec	TokenV3 JSON example contradicts its own base64 serialization (one hex digit)
O6	Comp	Keyset-ID residue collision $\times$ BIP32 $\times$ restore: the disclosed theft, plus our variant
O7	Impl	CVE-2025-65548: specification bounds the preimage at 32 bytes; enforcement was added only as a generic content-length limit
O8	Spec	2024 hash-to-curve domain separation absent (fixed silently; commit chain only)
O9	Spec	Protection-profile inversion: blind authentication (NUT-22) mandates DLEQ while value-bearing ecash leaves it optional
O10	Spec	Witness message-construction trap (escaped vs. unescaped secret), flagged only as a NOTE

is a failure of the blind-signature primitive alone. Second, the classes track detectability: specification-level findings show in specification review, implementation-level ones in code review, and compositional ones in cross-component modeling; the taxonomy thus tells maintainers where each fix must land. Third, two of the specification-level findings (†) are in the test vectors themselves, a reminder that conformance suites are part of the specification surface.

8 Cross-System Perspective

Do the nine properties describe Cashu, or deployed ecash generally? Table 6 applies them to two neighboring systems.

The P8 row carries the weight. Fedimint’s client derivation path is a hierarchy over *federation-id*, *module-id*, *key-type*, *amount*, and *index*: the issuer identity and even the note amount are domain separators, with the design document explicitly citing an attack (same blind nonce, different amount) that the amount

Table 6. Property outcomes across three deployed token systems. Evidence levels: executable (our implementation) vs. documentary (specification text).

Property	Cashu	Fedimint	Privacy Pass
P3 no inflation	StateOMU (this work)	tbs OMU (primitive line)	VOPRF one-more
P6 restore	violated norm (O3); formalized here	consensus-history (public, read-only)	replay vacuous (no recovery)
P8 derivation	V1 violated $\rightarrow$ V2 re-paired	design-level: federation isolation; path	federa- vacuous (random amount in nonces)
P9 cross-issuer iff theft	three conditions (Thm. 3)	federation isolation; interface unanalyzed	peg single-issuer trust
Evidence	executable	documentary + executable derivation check	documentary (RFCs)

component blocks. Our executable re-derivation of this scheme (HKDF-SHA512 per its specification) confirms federation-domain and amount-domain separation over 200/200 samples. The two ecosystems converged on the same property: Cashu reactively, after the theft; Fedimint proactively, at design time. Privacy Pass, which forgoes wallet recovery, has no deterministic-derivation surface at all: its P6/P8 are vacuous not by virtue but by omission. It pays for safety by giving up the feature.

Recoverability is not free. The comparison suggests a more general hypothesis: every capability to recover tokens from a seed arrives with new security obligations—a read-only recovery oracle, derivation binding across issuer identities, and cross-issuer theft prevention. Several signals point the same way: Cashu’s attack history, Fedimint’s design defenses, Privacy Pass’s omission, documented derivation hazards in adjacent projects, and an open Fedimint engineering thread on recovery, derivation, and reuse trade-offs.

9 Disclosure and Related Work

Disclosure. We report the two test-vector defects (O4/O5) to the specification maintainers concurrently with this submission, following the venue’s disclosure window; the repository has a history of vector-fix pull requests, and our drafts propose corrected values. O1–O3 and O9–O10 are published as specification observations with proposed MUST-level language (global spent domain; wallet-mandatory DLEQ). The public-key-claim variant requires no new window, since it analyzes an already-disclosed vulnerability class whose fixes exist and are deployed.

Deployed-protocol analyses. Our methodology follows the break–fix–verify tradition for deployed payment standards [10,11], transposed from card payments to a live ecash ecosystem, and pairs executable models with measurement and disclosure.

Subverted issuance. Concurrent work formalizes subversion-unlinkability for batch-issued credentials in the European digital-identity wallet (ePrint 2026/1229) and studies deployed anonymous-token systems in the same adversarial spirit. Our malicious-mint tier shares the subverted-issuer model; our target is integrity (theft) rather than privacy (tracing), our object is a deployed ecash system, and our defense analysis is complementary: DLEQ, the mechanism defeating our variant, is an instance (optional, hence practically subvertible) of the issuer-honesty proofs that line demands.

Primitive layer. The blind-signature literature covering Cashu’s core includes OMU gradations [6], threshold blind BLS and OPRFs [7], round-optimal TBS [8], and pairing-free constructions [9]; adjacent standardization includes Privacy Pass [12,13], rate-limited credentials (ARC), and anonymous credit tokens; metadata-bearing anonymous rate-limited tokens extend the subverted-issuer question. Fedimint’s issuance layer is specified as a threshold blind signature over BLS12-381, and threshold-issuance ecash has its own line of analysis [14]. None models keyset rotation, restore, or multi-mint identity; our P3 composes with, rather than replaces, these results.

10 Conclusion

We gave a system-level security analysis of Cashu, the dominant deployed Chaumian ecash system: a five-layer model with nine properties (three new), a stateful unforgeability theorem separating forgery from theft, a stronger public-key-claim variant of the disclosed attack, a mainnet measurement showing the sole remaining defense declared but unauthenticated, an iff characterization of cross-mint theft, and three replay-verified fixes. Three problems remain open: the peg interface of federated mints (where consensus finality meets probabilistic chain finality), the full theoremization of the recoverability obligations, and longitudinal re-measurement as the ecosystem adopts the fixes.

Data availability. All experiments are pure standard-library Python; source code, the per-mint measurement matrix, pinned specification snapshots, and scripts reproducing every number in this paper will be published upon acceptance.

References

1. cashumints.space mint index, fetched 2026-09-09/10.
2. Cashu NUT specifications. <https://github.com/cashubtc/nuts>
3. Conduition: Vulnerabilities in the Cashu ECash Protocol, Jan 2026. <https://conduition.io/code/cashu-disclosure/>
4. CVE-2025-65548: resource exhaustion via HTLC preimage; fixed in nutshell 0.18.0.
5. Nutshell repository commit chain, Feb–Mar 2024: domain-separator introduction and legacy h2c deprecation.

6. Lehmann, Nazarian, Özbay: Stronger Security for Threshold Blind Signatures. EU-ROCRYPT 2025. ePrint 2025/353.
7. Jarecki, Nazarian: Adaptively Secure Threshold Blind BLS Signatures and Threshold Oblivious PRF. ASIACRYPT 2025. ePrint 2025/483.
8. Fuchsbauer, Regen, Wee: Round-Optimal Threshold Blind Signatures without Random Oracles. CRYPTO 2026. ePrint 2026/433.
9. Chairattana-Apirom, Reichle, Tessaro: Playing Tag with Okamoto-Schnorr: Three-Move Pairing-Free Blind Signatures from DDH. CRYPTO 2026. ePrint 2026/571.
10. Basin, Sasse, Toro-Pozo: The EMV Standard: Break, Fix, Verify. IEEE S&P 2021.
11. Basin, Hofmeier, Sasse, Toro-Pozo: Getting Chip Card Payments Right. FM 2024.
12. Davidson, Iyengar, Wood: The Privacy Pass Architecture. RFC 9576, IETF 2024.
13. Pauly, Valdez, Wood: The Privacy Pass HTTP Authentication Scheme. RFC 9577, IETF 2024.
14. Rial, Piotrowska: Compact and Divisible E-Cash with Threshold Issuance. PoPETs 2023(4).